Project1_report_2022056562

: 2022056562 서민용

---

1. Camera System Design report requirement

- 제가 선택한 Camera view는 Orbit camera입니다.
- Oribit Camera와 Free Camera를 비교해보았을 때, 다음과 같은 이유로 Orbit을 선택했습니다.
  - 구현 난이도 측면에서,
  - Orbit Camera는 조금 더 어렵다고 생각했습니다.
    - Orbit action은 우선 기존 lab에서 사용하던 기법과 유사한 측면이 있어서 어려움을 덜 수 있겠지만,
      - 기존 lab에서는 eye point의 y 값에 대해서는 그냥 값을 linear 하게 늘리는 식으로 구현이 되어서, 완전한 Orbit action이 아닙니다.
      - 하여 구면좌표계를 구현해야하는데, 이는 확실히 구현이 더 어렵겠다는 생각이 들었습니다.
    - Pan action은 현재 eye와 center에 대한 벡터(수업에선 w라고 명시)를 구하고, 그 벡터의 방향에 따라서 eye point와 center point를 같은 거리만큼 옮겨주어야 하기에, 역시 구면좌표계를 구현해야합니다.
    - Zoom action 역시 Pan과 마찬가지로 벡터 w에 따라서 eye point를 옮겨주어야 하기에 복잡할 듯 합니다.
  - 그에 반해 Free Camera는
    - 다른 좌표계를 사용하지 않고 직교 좌표계를 사용하면 됩니다. 또한
    - Move action과 Jump action은 간단히 eye와 center를 xz 축 혹은 y축을 따라서 같은 거리만큼 옮겨주고,
    - Look around action은 오로지 center의 위치만 옮겨주면 되기에
  - Orbit Camera에 비해서 비교적 쉽게 구현할 수 있다고 판단했습니다.
  - 하지만 여러 기법을 배우고 익힘에 있어, 조물주와 같은 위치에서 관찰 할 수 있어야한다고 생각하여 Orbit Camera를 선택했습니다.
    - 또한 이동에 있어서 Orbit Camera가 훨씬 빠르고 정확하게 이동 할 수 있습니다.
    - 무엇보다 Free Camera는 Jump와 Look around를 통해서만 y축에 대해서 관여할 수 있습니다.
      이에 따라 높은 위치에서 결과물을 관찰하지 못한다는, 구현 난이도와 맞바꾸는 큰 Tradeoff 때문에 선택하지 않았습니다.

2. Interaction Design report requirement

2-* Perspective projection

- 코드에 대한 부분은 기존 lab 코드나 수업 때 다룬 부분과 동일하거나 이의 활용이라 명세에 적지 않았으나,
- Perspective projection에 대한 코드는 lab check 때 혼자서 구현해본 내용이기에 첨부합니다.

```
def framebuffer_size_callback(window, width, height):
    global g_P, g_cam_r
    glViewport(0, 0, width, height)
    per_height = 10.
    per_width = per_height * width/height
```

```
per_as = per_width/per_height
g_P = glm.perspective(45, per_as, 0.05, 2* g_cam_r)
```

- `5-viewport-fit-preserve-objratio-ortho.py` 와 흐름은 비슷하나, Perspective projection을 사용하기 위해서 `aspect` 값을 위와 같이 사용했습니다.

2-1 how the user controls the camera

- 사용자는 마우스와 키보드 모두 사용해서 camera를 제어합니다.
  - 허나 마우스 휠 input과 스마트폰에서 흔히 쓰이는 Zoom 제스처는 고려하지 않았습니다.
  - 또한 마우스 없이 노트북을 사용하는 user에게는 wheel input은 없는 input이고,
    노트북이 아닌 데스크탑을 사용하는 user에게는 Zoom 제스처는 없는 input이기 때문입니다.

2-2 how the camera changes in response to each user input

- Input : 마우스 드래그
  - 마우스 이동 방향의 반대 방향이 카메라 회전 방향이 됩니다.
  - Behavior : Orbit action

    > center point를 넘어서는 orbit 할 수 없도록, ϕ 의 값은 -90부터 90까지의 값만 가지도록 제한됩니다.
    > 그 이유는 90도 이상으로 넘어가게 되면, 카메라가 반전되어 상하 컨트롤이 정반대가 되어 불편하고 비직관적이기 때문입니다.

- Input : `X` 키를 누른 채 마우스 드래그
  - 마우스 이동 방향의 반대로 카메라가 평행 이동합니다.
  - Behavior : Pan action
- Input : `Z` 키를 누른 채 마우스 드래그
  - 마우스를 위로 올리면 Zoom in, 아래로 내리면 Zoom out 됩니다.
    - 트랙패드나 휴대폰, 패드에서 Zoom 제스처를 할 때를 고려해서 구현했습니다.
    - Zoom in 제스처는 두 손가락이 안에서 바깥으로 [수직 방향만 따졌을 때 위] 나가고,
    - Zoom out 제스처는 두 손가락이 바깥에서 안으로 [수직 방향만 따졌을 때 아래] 들어오기 때문에 직관적이라고 생각했습니다.
  - Behavior : Zoom action

    > Zoom action 시, 마우스 위치의 위 아래 움직임만 감지합니다.
    > 이는 양 옆으로 마우스가 움직일 때 Zoom이 일어나면 user에게 직관적인 interaction이 아니라고 생각했기 때문입니다.

    > 또한 r 값이 양수 값만 가지도록 제한됩니다.
    > 그 이유는 음수가 될 경우 grid 면을 뚫고 역시 카메라가 반전되어 상하 컨트롤이 정반대가 되기 때문입니다.
    > 또한 너무 Zoom in 되지 않도록 `g_cam_r` 의 최솟값이 0.1이 되도록 합니다.
    > [grid 면 아래에서의 object 관찰은 Orbit action을 통해서 이미 가능하기에, 겹치는 기능이라고 생각했기도 합니다.]

  - 또한 추가적으로, Zoom out 시 시야거리가 짧아 object가 rendering 되지 않는 부분을 보완하기 위해서, Projection matrix의 `far` 값은 `g_cam_r` 과 비례하게 업데이트 되도록 하였습니다.

```
# 이 코드는 while 문에 삽입되어 실시간으로 g_P를 업데이트 할 수있게 되어있습니다.
width, height = glfwGetWindowSize(window)
per_height = 10.
per_width = per_height * width/height
per_as = per_width/per_height
g_P = glm.perspective(45, per_as, 0.05, 2* g_cam_r)
```

  - 기존 lab이나 수업에서 다루지 않은 함수인 `glfwGetWindowSize(window)` 가 사용되었습니다.
- 과제 명세에 나와있는 https://threejs.org/examples/#misc_controls_orbit 의 방식을 많이 참고하였습니다.
- 마우스 input에 대해서는 lab1의 `2-glfw-input-handling.py` 를 참고하였습니다.
- 또한 `cmd`, `ctrl` 등의 특수키는 MacOS와 Window OS 간의 간극이 있을 듯 하여 사용하지 않았습니다.

2-3 why your interaction method is suitable

- 우선 lab5의 `3-lookAt.py` 에서 구현된 interaction은 키보드를 통한 Orbit action이라 할 수 있습니다.
- 과제에서 Orbit action에 대해서 키보드가 아닌 마우스 input을 선택한 이유는,
  - 사용자가 Camera가 이동하는 정도를 프로그램 내에서 설정 할 수 없기 때문에, 구조물을 둘러볼 때 불편함이 있습니다.
  - 또한 discrete 하게 Camera가 움직여지기 때문에, 구조물 관찰에 있어서 덜 interactive합니다.
- Pan과 Zoom action에 대해서 키보드 키 + 마우스 input을 선택한 이유는,
  - 과제 명세의 예시에서도 그렇고 원래 익숙한 조작법이기도 했고,
  - 2-1 섹션에서 언급 했듯 노트북과 데스크탑 user 모두가 쉽게 사용할 수 있기 때문입니다.

2-4 any design considerations or trade-offs

- grid와 체스판을 그릴 때, 기존에 쓰이던 frame의 vao를 재활용 했습니다.
  - lab06의 `1-ortho.py` 에서 cube를 여러개 만드는 코드에서 착안하여 구현하였습니다.

```
def draw_check(vao, MVP, loc_MVP):
    glBindVertexArray(vao)
    for q in range(5):
        for w in range(5):
            MVP_check = MVP * glm.translate(glm.vec3(1*q, 0, 1*w))* glm.sc
ale(glm.vec3(.5,.5,.5))
            glUniformMatrix4fv(loc_MVP, 1, GL_FALSE, glm.value_ptr(MVP_che
ck))
            glDrawArrays(GL_TRIANGLES, 0, 24)

def draw_grid(vao, MVP, loc_MVP):
    glBindVertexArray(vao)
    scale = 3
    MVP_ = MVP * glm.scale(glm.vec3(scale,scale,scale))
    for q in range(-8,7):
        for w in range(-7,8):
            MVP_grid = MVP_ * glm.translate(glm.vec3(1*q/scale, 0, 1*w/sca
le))
            glUniformMatrix4fv(loc_MVP, 1, GL_FALSE, glm.value_ptr(MVP_gri
```

```
d))
                glDrawArrays(GL_LINES, 0, 4)
```

  - 추가적으로 grid에 대한 vao를 따로 만들지 않아도 되고, 체스판을 그릴 때도 최소한의 vao를 사용했기 때문에 효율적이라고 생각합니다.
- Zoom action에 대해서, 키보드 + 마우스 드래그라는 interaction이 조금 어색하다고 생각합니다.
  - 초기에 생각할 때, 수직방향의 슬라이드 바를 상상하고 만들었는데, window 내에서 마우스가 움직일 때 좌우방향으로는 고정이 안되기 때문에, 조작에 어색함이 있습니다.
    - 키보드로만 Zoom action을 한다면, 직관적이기도 하고 조작이 편리합니다.
  - 추후 프로젝트에서 구현할 때, `z` 키를 누른 상태라면 마우스 포인터 조작이 좌우로는 되지 않게 만들 수 있다면 이와 같이 개선하는게 좋겠습니다.
- 또한 모든 action은 마우스 드래그가 동반되는데, 이는 편리하긴 하지만 사용자 별로 체감이 다를 수 있다는 여지가 있습니다.
  - 사용자들은 각자 다른 트랙패드, 마우스에 대해서 다른 마우스 민감도를 가질테니 각자의 체감 민감도가 다를 것입니다.
  - 추후 프로젝트에서 키보드를 통해 각 action의 민감도 조절 기능이 있으면 좋을 듯 합니다.

3. Video link

- https://youtu.be/8I0z-jyUleo?si=IzzdOQUbytgaTUia